

TECH

TYPESCRIPT

BASICS / TYPES GENERICS / ADVANCED TS TS ARCHITECTURE

TS ARCHITECTURE

15 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК typescriptlang.org/docs

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	SHARED TYPES Q1 · Q3	DTO	ДУБЛИРОВАНИЕ	Где должны жить общие типы и как не плодить копии.
ДЕНЬ 2 ТЯЖЁЛЫЙ	СИНХРОНИЗАЦИЯ FE / BE Q2			Главный источник багов в TS-проектах. Решений несколько – OpenAPI, GraphQL, tRPC, monorepo-shared, codegen.
ДЕНЬ 3 СРЕДНИЙ	RUNTIME ВАЛИДАЦИЯ o-ts Q4 · Q5	Zod / Valibot / i		TS исчезает после компиляции – приходит Zod.
ДЕНЬ 4 СРЕДНИЙ	DOMAIN vs API MODEL Q6			Почему модель приложения и модель API – это часто разные типы.
ДЕНЬ 5 ТЯЖЁЛЫЙ	МИГРАЦИИ Q7 · Q8	Flow → TS	STRICT MODE	Как ввести типы в большой проект без даяунтайма.
ДЕНЬ 6 СРЕДНИЙ	as REVIEW Q9 · Q10 · Q11	ЧТО ТАКОЕ ПЛОХОЙ TS		Code-quality. Что считать антипаттерном и как ловить.
ДЕНЬ 7 СРЕДНИЙ	BACKWARD-COMPAT S REFACTOR Q12 · Q13 · Q14 · Q15	LEGACY	STRICTNES	Большие проекты: совместимость API, лего, refactoring под защитой типов.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 2 ВОПРОСА

SHARED TYPES DTO ДУБЛИРОВАНИЕ

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Где должны жить общие типы и как не плодить копии.

ВОПРОСЫ ДНЯ

Q01 Где должны жить shared types?

Q03 Как избежать дублирования DTO?

Где должны жить shared types?

ОПИСАНИЕ

Shared types – типы, используемые в нескольких частях приложения (frontend, backend, shared lib). Хранение должно решать две проблемы: единственный источник правды (SSOT) и доступность из всех точек. Распространённые решения – отдельный пакет в monorepo, общий npm-пакет, генерация из спецификации (OpenAPI, GraphQL, protobuf).

КАК РАБОТАЕТ

- 01 Monorepo: пакет packages/types или packages/shared, на который ссылаются и FE, и BE.
- 02 OpenAPI/GraphQL/tRPC: специальный код-ген производит типы автоматически.
- 03 Protocol Buffers / FlatBuffers: спецификация в .proto/.fbs, генерация типов на каждый язык.
- 04 Внешний npm-пакет (если monorepo нет): @company/types.
- 05 Главное правило: типы НЕ дублируются вручную в разных репозиториях.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 FE-BE взаимодействие (DTO).
- 02 Несколько фронтов на разных стеках (Web + RN).
- 03 Микросервисы, общающиеся между собой.
- 04 SDK, выпускаемые наружу.

ПОДВОДНЫЕ КАМНИ

- 01 Типы зависят от runtime-кода – пакет тяжелеет.
- 02 Версионирование между monorepo-пакетами требует дисциплины.
- 03 OpenAPI с расхождениями spec ↔ implementation = баги.
- 04 Чрезмерное «общение» типов – высокая связанность.

КОД

// Структура monorepo

```

/repo
├── packages
│   ├── types      ← shared types
│   ├── ui-kit
│   ├── frontend   ← imports @repo/types
│   └── backend     ← imports @repo/types

```

// Codegen из OpenAPI

```

// scripts/gen-types.ts
// openapi-typescript ./api.yaml -o ./packages/types/api.ts

```

МОГУТ ДОСПРОСИТЬ

- 01 Какие минусы у external npm-пакета по сравнению с моногеро?
- 02 Чем кодоген хуже ручных типов?
- 03 Где описывать типы, которые нужны только FE?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/project-references.html>

Как избежать дублирования DTO?

ОПИСАНИЕ

DTO часто дублируется между FE, BE и базой. Решение – единый источник правды и трансформации (mapping) от него к специфичным моделям. Можно или генерировать DTO из спецификации API (OpenAPI, GraphQL, ts-rest, tRPC), или хранить его в shared-package. Каждое слой может иметь свою модель, отображаемую через явные адаптеры.

КАК РАБОТАЕТ

- 01 SSOT: одна спецификация (OpenAPI/GraphQL/Zod-схема).
- 02 Кодоген: генерация TS-типов на FE и BE.
- 03 Domain-model на FE и BE отделена от DTO; адаптер dto-domain.
- 04 Тесты: smoke-тест на парсинг каждой формы DTO Zod-схемой.
- 05 При смене DTO ломается код на этапе компиляции – это цель.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Frontend и Backend на TypeScript – кодоген.
- 02 Multi-tenant API с десятками моделей.
- 03 Микросервисы – спека описывает контракт.
- 04 SDK-разработка.

ПОДВОДНЫЕ КАМНИ

- 01 Кодоген генерирует «голый» тип – без бизнес-логики, поэтому domain-model отдельно.
- 02 Без тестов на маппинг DTO ↔ domain рассинхронизация «проползает».
- 03 Большие OpenAPI-спеки рожают мегафайлы типов – неудобно для IDE.

КОД

// tRPC – типы автоматически

```
// server.ts
export const appRouter = t.router({
  user: t.procedure.query(() => ({ id: '1', name: 'A' })),
});
export type AppRouter = typeof appRouter;

// client.ts
import type { AppRouter } from '../server';
const trpc = createTRPCProxyClient<AppRouter>({ ... });
// trpc.user.query() полностью типобезопасно
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем кодоген лучше ручных типов?
- 02 Где должен жить адаптер DTO→domain?
- 03 Что делать, если бэк меняется чаще API-спеки?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

СИНХРОНИЗАЦИЯ FE / BE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Главный источник багов в TS-проектах. Решений несколько – OpenAPI, GraphQL, tRPC, mono repo-shared, codegen.

ВОПРОСЫ ДНЯ

Q02 Как синхронизировать типы frontend и backend?

ВОПРОС Q02

Как синхронизировать типы frontend и backend?

ОПИСАНИЕ

Главный риск — рассинхронизация: FE и BE работают с разными типами одного и того же объекта. Решений несколько, выбор зависит от стека: shared-package в monorepo, OpenAPI/Swagger + кодоген, GraphQL + codegen, tRPC, ts-rest, общая Zod-схема. У каждого свои компромиссы между точностью, удобством и стоимостью внедрения.

КАК РАБОТАЕТ

- 01 Monorepo + shared package: типы пишутся один раз и импортируются в FE и BE.
- 02 OpenAPI: бэк публикует /openapi.json, фронт через openapi-typescript генерирует типы.
- 03 GraphQL: схема — спека, codegen на FE, типизированные operations.
- 04 tRPC: бэк определяет router, фронт импортирует его тип (не значение). Все runtime — через RPC.
- 05 Zod-схема: и валидация ввода, и тип через z.infer<>. На FE и BE.
- 06 ts-rest: декларация контракта в общем месте, FE и BE получают одинаковые типы.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой fullstack TS-проект.
- 02 Команды с monorepo (Turborepo, Nx).
- 03 GraphQL-приложения с codegen.
- 04 tRPC — отлично для small-medium fullstack.
- 05 Микросервисы с общей API-схемой (OpenAPI + codegen).

ПОДВОДНЫЕ КАМНИ

- 01 OpenAPI-генератор может выдавать «грубые» типы (any в ответе ошибки и т.п.) — нужен тюнинг.
- 02 tRPC привязывает FE к BE — труднее versioning.
- 03 Shared package требует синхронизации версий.
- 04 Несовместимости versions FE и BE могут проскользнуть, если контракт не зафиксирован в спеке.
- 05 Без runtime-валидации даже синхронизированные типы могут врать (бэк отдаёт другое).

КОД

```
// OpenAPI codegen
// package.json
// "gen": "openapi-typescript ./api.json -o ./api.ts"
import type { paths } from './api';
```

```
type GetUserResp = paths['/users/{id}']['get']['responses']['200']['content']['application/json'];
```

// tRPC

```
// shared types – через typeof router  
import type { AppRouter } from '../server';  
const trpc = createTRPCProxyClient<AppRouter>({ ... });
```

// Zod-схема

```
const UserSchema = z.object({ id: z.string(), name: z.string() });  
type User = z.infer<typeof UserSchema>;  
// одна схема – два эффекта: тип + валидация
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда выбрать tRPC, а когда OpenAPI?
- 02 Почему синхронизированные типы не отменяют валидацию?
- 03 Какие типы генерируют codegen из GraphQL?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/project-references.html>

ДЕНЬ 3

НАГРУЗКА

СРЕДНИЙ

2 ВОПРОСА

RUNTIME ВАЛИДАЦИЯ**Zod / Valibot / io-ts****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

TS исчезает после компиляции — приходит Zod.

ВОПРОСЫ ДНЯ

Q04 Как валидировать runtime данные, если TypeScript исчезает после компиляции?

Q05 Когда нужен Zod / Valibot / io-ts?

Как валидировать runtime данные, если TypeScript исчезает после ко

ОПИСАНИЕ

TS — это compile-time проверка. Во время выполнения никаких типов нет; данные, приходящие извне (API, localStorage, JSON.parse), могут не соответствовать заявленному типу. Решение — отдельный слой runtime-валидации: библиотеки Zod, Valibot, io-ts, Yup, Joi, или ручные type guards. Валидация даёт и проверку, и тип через type-inference.

КАК РАБОТАЕТ

- 01 Schema-first: описать схему → получить тип через z.infer.
- 02 Парсинг: schema.parse(data) бросает на ошибке, schema.safeParse возвращает Result.
- 03 type guards для простых случаев: function isUser(x): x is User { ... }.
- 04 io-ts работает в стиле fp-ts с Either-результатом.
- 05 Можно интегрировать с react-hook-form через resolver, с tRPC — через input validator.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JSON.parse / fetch / localStorage — везде, где данные приходят извне.
- 02 Submit формы — валидация перед отправкой.
- 03 Postmessage / WebSocket — проверка протокола.
- 04 ENV-variables — валидация на старте (z.object({ DB_URL: z.string() })).

ПОДВОДНЫЕ КАМНИ

- 01 Валидация — стоит CPU. На горячих путях (сотни запросов в секунду) можно делать выборочно.
- 02 Дублирование TS-тип vs Zod-schema — антипаттерн. Лучше schema-first.
- 03 Любая валидация устаревает с изменением реального API.
- 04 io-ts/fp-ts — функциональный стиль, кривая обучения.

КОД

// Zod — schema-first

```
import { z } from 'zod';
const UserSchema = z.object({
  id: z.string(),
  name: z.string(),
  email: z.string().email().optional(),
});
type User = z.infer<typeof UserSchema>;

function loadUser(raw: unknown): User {
```

```
    return UserSchema.parse(raw); // throws на ошибке
  }
```

// safeParse – Result-стиль

```
const r = UserSchema.safeParse(raw);
if (!r.success) console.log(r.error.format());
else use(r.data);
```

МОГУТ ДОСПРОСИТЬ

- 01 Где валидировать – на FE или на BE?
- 02 Что лучше – type guard руками или Zod?
- 03 Как валидировать поток (stream) большим объёмом данных?

ПОЛНАЯ СТАТЬЯ

<https://zod.dev/>

Когда нужен Zod / Valibot / io-ts?

ОПИСАНИЕ

Зависит от роли валидации в проекте. Zod – самый популярный: универсальный schema-first, удобный API, хорошая интеграция с react-hook-form, tRPC. Valibot – новее, легковеснее, tree-shake-friendly. io-ts – функциональный, требует знания fp-ts. Yup/Joi – альтернативы Zod без TS-инференса (старые).

КАК РАБОТАЕТ

- 01 Zod: chain-API (z.string().min(1).email()), JSON-schema поддержка, partial/refine/transform.
- 02 Valibot: тот же концепт, меньший bundle. Хорош для edge-runtime.
- 03 io-ts: codecs Either-результат, fp-ts экосистема.
- 04 У всех есть z.infer<typeof Schema> – один источник для типа и валидации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Zod – дефолт для большинства проектов.
- 02 Valibot – когда важен размер бандла (Edge / SDK).
- 03 io-ts – fp-проекты с Either / Task.
- 04 Yup – старый legacy-проект (заменять на Zod при возможности).

ПОДВОДНЫЕ КАМНИ

- 01 Zod 4 поломал ряд API из 3 – проверять migration guide.
- 02 Yup-инференс часто кривой – Zod лучше.
- 03 io-ts требует fp-ts: повышает порог входа в команду.
- 04 Большие схемы Zod увеличивают bundle.

КОД

// Zod – типичный пример

```
const Form = z.object({
  name: z.string().min(1),
  age: z.number().int().min(18),
  email: z.string().email().optional(),
});
type Form = z.infer<typeof Form>;
```

// Valibot

```
import * as v from 'valibot';
const Form = v.object({
  name: v.string([v.minLength(1)]),
  age: v.number([v.integer(), v.minValue(18)]),
```

```
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие минусы у Zod 4?
- 02 Что выбрать для edge-runtime (Vercel/Cloudflare)?
- 03 Чем `z.refine` отличается от `z.transform`?

ПОЛНАЯ СТАТЬЯ

<https://zod.dev/>

DOMAIN vs API MODEL

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Почему модель приложения и модель API — это часто разные типы.

ВОПРОСЫ ДНЯ

Q06 Как отличать domain model от API model?

ВОПРОС Q06

Как отличать domain model от API model?

ОПИСАНИЕ

Domain model – то, как объект выглядит ВНУТРИ приложения (после загрузки и трансформаций). API model – формат, в котором объект приходит/уходит по сети. Они часто расходятся: даты как ISO-строки в API ↔ Date в domain; snake_case в API ↔ camelCase в domain; flattened ↔ nested. Хорошая практика – явно разделить и иметь адаптер.

КАК РАБОТАЕТ

```
01 type ApiUser = { id: string; created_at: string };
02 type DomainUser = { id: string; createdAt: Date };
03 Адаптер: function fromApi(a: ApiUser): DomainUser.
04 Адаптер: function toApi(d: DomainUser): ApiUser (для сохранения).
05 Тестируется парой round-trip: toApi(fromApi(x)) ≈ x.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Любая работа с REST/GraphQL.
02 Множественные источники данных (несколько API).
03 Версионированный API – domain не меняется при смене API-версии.
04 Тестируемая бизнес-логика – без зависимости от формата сети.
```

ПОДВОДНЫЕ КАМНИ

```
01 Прямое использование ApiUser в UI = протекание внешних соглашений в
product-логику.
02 Без адаптера баги «даты как строки» вылезают по всему коду.
03 Big-bang перевод domain-моделей в legacy-проектах – много кода. Делают по чуть.
```

КОД

// Адаптер dto-domain

```
type ApiUser = { id: string; created_at: string };
type DomainUser = { id: string; createdAt: Date };

function fromApi(a: ApiUser): DomainUser {
  return { id: a.id, createdAt: new Date(a.created_at) };
}
function toApi(d: DomainUser): ApiUser {
  return { id: d.id, created_at: d.createdAt.toISOString() };
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Где должен жить адаптер – в API-слое или в domain?
- 02 Как протестировать round-trip?
- 03 Что делать с API, у которого формат варьируется?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/objects.html>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

МИГРАЦИИ **Flow → TS** **STRICT MODE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Как ввести типы в большой проект без даунтайма.

ВОПРОСЫ ДНЯ

Q07 Как организовать migration с Flow на TypeScript?

Q08 Как вводить strict mode в большом проекте?

Как организовать migration с Flow на TypeScript?

ОПИСАНИЕ

Миграция с Flow – большой инкрементальный проект. Стратегия: рядом с Flow поднять TS, разрешить смешение (JS + Flow + TS), переводить файлы по одному, начиная с листьев dependency-графа. Helper-инструменты: flow-to-ts, ручные правки. Цель – не сломать билд, не остановить разработку.

КАК РАБОТАЕТ

- 01 Включить TS в build: tsconfig с allowJs + checkJs=false.
- 02 Перевести один лист (компонент без внутренних импортов Flow-only).
- 03 Запускать flow-to-ts (или аналог) для рутины – типы Flow перевести в TS-синтаксис.
- 04 Постепенно отключать Flow на переведённых файлах.
- 05 В конце удалить Flow и связанные babel-плагины.
- 06 Сложные generics, \$Keys, \$ObjMap часто требуют ручной адаптации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие React-проекты, исторически писавшиеся на Flow.
- 02 Команды с подавляющим выбором TS в новых проектах.
- 03 Проекты, где экосистема Flow перестала развиваться.

ПОДВОДНЫЕ КАМНИ

- 01 Конвертеры дают «ленивые» типы – нужна ручная доработка.
- 02 Flow и TS по-разному narrow'ят union – поведение может поменяться.
- 03 Flow-only зависимости (некоторые npm-пакеты) – найти TS-альтернативу или написать .d.ts.
- 04 Параллельный билд (Flow + TS) тяжелее CI.

КОД

// Tsconfig для inkrement-миграции

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": false,
    "noEmit": true,
    "strict": false
  }
}
// потом ужесточать постепенно
```

МОГУТ ДОСПРОСИТЬ

- 01 Где начать миграцию?
- 02 Что делать с Flow-only зависимостями?
- 03 Сколько времени реально занимает миграция большого FE?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/migrating-from-javascript.html>

Как вводить strict mode в большом проекте?

ОПИСАНИЕ

Включать strict сразу – обычно вызывает 1000+ ошибок и пугает команду. Лучше идти step-by-step: включать конкретные флаги (noImplicitAny, strictNullChecks, strictFunctionTypes...), фиксировать ошибки, продолжать. Можно использовать `// @ts-expect-error` / `@ts-strict-ignore` временно, с TODO-комментариями.

КАК РАБОТАЕТ

- 01 Самые тяжёлые: `strictNullChecks` (null/undefined), `noImplicitAny` (явные any).
- 02 Включать по одному: `noImplicitAny` → `strictNullChecks` → `strictFunctionTypes` → `strictPropertyInitialization` → `strictBindCallApply` → `noImplicitThis` → `alwaysStrict`.
- 03 Использовать `// @ts-expect-error` для явных временных лазеек (в отличие от `@ts-ignore` – не молчит, если ошибка исчезла).
- 04 `tsconfig` можно делать модульно: `strict-tsconfig.json` для новых пакетов, расслабленный – для legacy.
- 05 Метрика – счётчик "any" в проекте, должен снижаться.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой проект, где TS уже подключён, но strict выключен.
- 02 Большие React-приложения, где `strictNullChecks` дал бы тысячи ошибок.
- 03 При onboarding нового кода – он должен соответствовать strict.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много `@ts-expect-error` превращается в чёрный список – нужен мониторинг.
- 02 Включение strict без plan'a → команда тонет, релизы тормозят.
- 03 `noImplicitAny` ловит много старых any → бывает сложно корректно протипизировать legacy.
- 04 Часть зависимостей может терять типы при strict – проверять.

КОД

// Поэтапное включение

```
// step 1
{
  "compilerOptions": { "noImplicitAny": true }
}
// step 2 (после фикса)
{
  "compilerOptions": {
    "noImplicitAny": true,
    "strictNullChecks": true
  }
}
```

```
}  
}
```

// @ts-expect-error vs @ts-ignore

```
// @ts-expect-error TODO: fix after refactor  
doSomethingBad();
```

МОГУТ ДОСПРОСИТЬ

- 01 В каком порядке включать strict-флаги?
- 02 Чем @ts-expect-error лучше @ts-ignore?
- 03 Как мониторить counter «any» в проекте?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/tsconfig#strict>

as**REVIEW****ЧТО ТАКОЕ ПЛОХОЙ TS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Code-quality. Что считать антипаттерном и как ловить.

ВОПРОСЫ ДНЯ

Q09 Как бороться с `as` по всему проекту?

Q10 Как ревьюить TypeScript-код?

Q11 Какой TS-код ты считаешь плохим?

ВОПРОС 009

Как бороться с `as` по всему проекту?

ОПИСАНИЕ

`as` – type assertion, который говорит компилятору «доверься мне». В отличие от `type guard`, ничего не проверяется – просто меняется тип. Часто это лазейка, скрывающая реальную проблему типизации. Стратегия – найти все `as`, разделить «оправданные» и «костыль», постепенно убрать костыли через типгварды, дженерики или `Zod`-валидацию.

КАК РАБОТАЕТ

- 01 `x as T` – type assertion, runtime ничего не делает.
- 02 `as const` – литеральная заморозка, обычно ок.
- 03 `as unknown as T` – двойное приведение, явно сигнализирует «я знаю, что делаю».
- 04 Лекарства: `type guards`, `generic-функции`, `Zod-схемы`, `narrowing` через `discriminated union`.
- 05 Метрика: `grep ' as '` в кодовой базе.
- 06 Линтер: `@typescript-eslint/consistent-type-assertions` с правилами `no-as-only`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code-review – каждый новый `as` в PR должен быть оправдан.
- 02 Audit legacy – посмотреть `top` файлы по количеству `as`.
- 03 Заменять `as` на `type guard` / `Zod` / `generic`.
- 04 В тестах `as DeepPartial<T>` – допустимо, в `production` коде – обычно нет.

ПОДВОДНЫЕ КАМНИ

- 01 `as any` – почти всегда антипаттерн.
- 02 `as unknown as T` – явно говорит «знаю, что делаю», но всё ещё рискованно.
- 03 `as` без рефакторинга маскирует баги: `typescript` молчит, а `runtime` падает.
- 04 Заглушки в legacy «`as User`» пропускают невалидные объекты.

КОД

// Плохо

```
const data = JSON.parse(raw) as User; // нет проверки
const el = document.getElementById('x') as HTMLInputElement; // без проверки на null
```

// Лучше

```
const data = UserSchema.parse(JSON.parse(raw));

const el = document.getElementById('x');
if (el instanceof HTMLInputElement) {
  // el – HTMLInputElement
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `as` – оправдано?
- 02 Чем `satisfies` отличается от `as`?
- 03 Как линтером запретить `as any`?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#type-assertions>

ВОПРОС Q10

Как ревьюить TypeScript-код?

ОПИСАНИЕ

TS-ревью — проверка не только бизнес-логики, но и качества типов. Что смотреть: `any` и `as` без обоснования; переусложнённые `generic`; смешивание `domain` и `API` моделей; отсутствие `runtime`-валидации входных данных; неиспользуемые поля; устаревшие `@ts-ignore`; правильное использование `utility-types`.

КАК РАБОТАЕТ

- 01 Чек-лист: `any`, `as`, `@ts-ignore` — каждый требует обоснования.
- 02 `Generic`-параметры — простые, понятные, минимум.
- 03 Сужение `union` — через `narrowing`, не как `assertion`.
- 04 `Discriminated union` — везде, где разнотипные варианты.
- 05 `Runtime`-валидация — обязательна для `unknown`-входов.
- 06 Соответствие `naming`-конвенциям проекта.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR с TS-кодом.
- 02 `Onboarding` новых членов команды.
- 03 `Audit` перед релизом или рефакторингом.
- 04 `Code-quality` дашборды и автоматические правила.

ПОДВОДНЫЕ КАМНИ

- 01 Ревьюер фокусируется только на логике, упуская типы.
- 02 Жёсткое соблюдение «никаких `any`» в спорных местах тормозит PR.
- 03 «Умные» PR с многоуровневыми `conditional`-типами трудно ревьюить.
- 04 Принципы ревью должны быть документированы — иначе разные ревьюеры противоречат друг другу.

КОД

// Чек-лист

```
// 1. any? → unknown + guard или generic
// 2. as? → guard / Zod / narrowing
// 3. сложный generic? → понятен ли?
// 4. discriminated union? → есть exhaustive check?
// 5. unknown-вход? → runtime-валидация?
// 6. dead code? → удалить
```

МОГУТ ДОСПРОСИТЬ

- 01 Что чаще всего пропускают TS-ревьюеры?

02 Как автоматизировать часть проверок?

03 Где хранить TS-стайлгайд проекта?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

ВОПРОС Q11

Какой TS-код ты считаешь плохим?

ОПИСАНИЕ

Плохой TS-код – это код, в котором типы либо «врут», либо перегружают читателя без пользы. Маркеры: `any/as` везде; 5-этажные `generic`; типы, описанные двойным путём (TS-тип и `Zod`); разница `domain/API` игнорируется; `@ts-ignore` без комментария; `mutable global state`; `impossible states` представимы.

КАК РАБОТАЕТ

- 01 Антипаттерны: повсеместный `any`, `as User` без проверки, `@ts-ignore` без `TODO`.
- 02 Перегруз: `generic`, который никто не понимает, `mapped+conditional` на 10 строк.
- 03 Дублирование: TS-тип отдельно от `Zod`-схемы, они расходятся.
- 04 Слабые контракты: API-ответы без `runtime`-валидации.
- 05 Smell: `?:` на каждом поле – модель не моделирует ничего.
- 06 Smell: `any[]` вместо `typed array`; `never` вместо отсутствия обработки кейса.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review – давать структурированный `feedback`.
- 02 Onboarding – показывать примеры «как НЕ делать».
- 03 Аудит `legacy`.

ПОДВОДНЫЕ КАМНИ

- 01 Ярлыки «плохой код» без альтернативы – деморализуют.
- 02 Иногда «плохой» код – это `compromise`-решение под временной `constraint`.
- 03 Без культуры ревью нет договорённости, что считать плохим.

КОД

// Антипаттерны

```
// плохо
function loadUser(id: any): any { ... }
const u = data as User;
// @ts-ignore
doStuff();
type User = {
  id?: string;
  name?: string;
  email?: string;
  // всё опционально – модель ничего не гарантирует
};
```

МОГУТ ДОСПРОСИТЬ

- 01 Как договариваться о «плохом TS» в команде?
- 02 Чем `grep '@ts-ignore'` помогает?
- 03 Где культурно полагать `any`?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

ДЕНЬ 7 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

BACKWARD-COMPAT**LEGACY****STRICTNESS****REFACTOR****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Большие проекты: совместимость API, легаси, refactoring под защитой типов.

ВОПРОСЫ ДНЯ

- Q12 Как проектировать типы для backwards-compatible API?
- Q13 Как типизировать legacy code?
- Q14 Какой уровень strictness ты считаешь must-have?
- Q15 Как TS помогает в refactoring?

Как проектировать типы для backwards-compatible API?

ОПИСАНИЕ

Backwards-compatible API – старые клиенты должны продолжать работать с новой версией. Ключевые правила: не удалять и не переименовывать поля; добавлять новые только как optional; для breaking changes – версионировать; discriminated union с version-полем – паттерн для evolution. Внутри проекта – то же самое для shared types.

КАК РАБОТАЕТ

- 01 Add: новое поле – обязательно optional (?:).
- 02 Remove: deprecated сначала, потом – major-version-bump.
- 03 Rename: добавить новое имя как optional, оставить старое, потом – major-version-bump.
- 04 Discriminated union по version-полю позволяет читать несколько версий одновременно.
- 05 Tools: api-extractor для библиотек, semver-checks для CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Публичные npm-библиотеки.
- 02 REST/GraphQL/gRPC API.
- 03 Long-running SaaS, где клиенты подключаются годами.
- 04 WebSocket-протоколы реалтайм-сервисов.

ПОДВОДНЫЕ КАМНИ

- 01 Поле «неиспользуется» – не основание удалить (старый клиент может всё ещё писать его).
- 02 Optional поля без default-значений – клиенты могут не знать, какое поведение по-умолчанию.
- 03 Слишком много версий протокола одновременно – сложно поддерживать.
- 04 TS-инструменты не ловят breaking change в runtime поведении (поменяли семантику поля).

КОД

// Версионирование протокола

```
type V1 = { version: 1; data: string };
type V2 = { version: 2; data: string; meta: Meta };
type AnyMsg = V1 | V2;

function handle(m: AnyMsg) {
  switch (m.version) {
    case 1: handleV1(m); break;
```



```
    case 2: handleV2(m); break;
  }
}
```

// Расширение API без breaking

```
type ApiV1 = { id: string; name: string };
type ApiV2 = ApiV1 & { email?: string }; // optional!
```

МОГУТ ДОСПРОСИТЬ

- 01 Что считать breaking change в TS-типе?
- 02 Как поддерживать несколько версий одновременно?
- 03 Какие tools ловят breaking?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

Как типизировать legacy code?

ОПИСАНИЕ

Legacy JS-код без типов – типизировать постепенно, не пытаясь сразу сделать всё идеально. Стратегия: подключить TS, разрешить `allowJs`, написать `.d.ts` на вход/выход модулей, переводить файл за файлом. Use `any/unknown` как временную меру с `TODO`. Ошибки оценивать пакетами – иначе утонуть.

КАК РАБОТАЕТ

- 01 `tsconfig: allowJs, checkJs` (по выбору), `noEmit` для CI-проверки.
- 02 `.d.ts`-файлы рядом с `.js` – лучший способ ввести типы без рефакторинга.
- 03 Прогресс отслеживать через метрики: `% .ts vs .js`, количество `any`.
- 04 В первую очередь типизировать API-границы и `shared utility`, не внутреннюю кухню.
- 05 `Strict-mode` – ПОТОМ, после базовой типизации.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 JS-проекты, переходящие на TS.
- 02 Старые библиотеки без `@types`.
- 03 Микросервис на старом стеке.
- 04 Любой кодовый юнит, где приходится взаимодействовать с legacy.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang миграция – проект тонет.
- 02 `any`-езде – победили компилятор, потеряли пользу TS.
- 03 TS-плагины бандлера могут сломать build при внезапном `checkJs=true`.
- 04 Legacy-зависимости часто плохо типизированы – переписать `.d.ts` и опубликовать в `DefinitelyTyped`.

КОД

```
// .d.ts рядом с .js

// utils.js
module.exports = { add(a, b) { return a + b; } };

// utils.d.ts
declare module './utils' {
  export function add(a: number, b: number): number;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 С чего начать типизацию legacy?

02 Как считать ROI миграции?

03 Что писать в .d.ts на чужой пакет?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/migrating-from-javascript.html>

Какой уровень `strictness` ты считаешь `must-have`?

ОПИСАНИЕ

Минимально-нужный уровень – `strict: true` (включает все `strict`-флаги). Это база, без которой TS не сильно полезнее JSDoc. Дополнительно для `production`-приложений: `noUncheckedIndexedAccess` (доступ по индексу даёт `T | undefined`), `exactOptionalPropertyTypes`, `noFallthroughCasesInSwitch`, `noImplicitOverride`.

КАК РАБОТАЕТ

- 01 `strict: true` – собирает `noImplicitAny`, `strictNullChecks`, `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, `alwaysStrict`, `noImplicitThis`, `useUnknownInCatchVariables`.
- 02 `noUncheckedIndexedAccess` – частый `upgrade`, ловит много багов с массивами.
- 03 `exactOptionalPropertyTypes` – различает «нет ключа» и «ключ=`undefined`».
- 04 `noImplicitReturns`, `noFallthroughCasesInSwitch` – не входят в `strict`, но полезны.
- 05 `noImplicitOverride` – для классов с `extends`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой новый TS-проект – `strict` сразу.
- 02 Audit и обновление `tsconfig` в существующих проектах.
- 03 Onboarding `tsconfig` в монорепо/библиотеках.

ПОДВОДНЫЕ КАМНИ

- 01 `noUncheckedIndexedAccess` раздражает на массивах – придётся везде проверять.
- 02 `exactOptionalPropertyTypes` ломает API, где `attaches undefined` вместо отсутствия поля.
- 03 Слишком много флагов – время CI растёт.
- 04 Зависимости без типов – приходится писать `.d.ts`.

КОД

```
// Production-tsconfig
```

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noFallthroughCasesInSwitch": true,
    "noImplicitOverride": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  }
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `noUncheckedIndexedAccess` полезен на массивах?
- 02 Что меняет `exactOptionalPropertyTypes` в API?
- 03 Какие флаги вы бы добавили вне `strict`?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/tsconfig#strict>

Как TS помогает в refactoring?

ОПИСАНИЕ

TS превращает рефакторинг из «надеемся, что не сломаем» в «компилятор покажет всё, что отвалилось». Изменили тип – каждое использование, не соответствующее новому контракту, становится ошибкой. Это работает на уровне всего проекта – IDE показывает все места.

КАК РАБОТАЕТ

- 01 Find references / Rename Symbol в IDE – type-aware, точный.
- 02 Изменили сигнатуру функции – все вызовы сразу видны.
- 03 Переименовали поле в типе – все usage подсвечены.
- 04 Discriminated union: добавили новый кейс – exhaustive check ловит везде.
- 05 Optional → required: компилятор показывает, кто не передаёт поле.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Архитектурные рефакторинги (extract module, rename API).
- 02 Правка контрактов между FE и BE.
- 03 Миграции (на новую версию библиотеки, на новый формат state).
- 04 Автоматизация через codemod-инструменты (jscodeshift, ts-morph).

ПОДВОДНЫЕ КАМНИ

- 01 Без strict-mode рефакторинг ловит меньше – any съедает ошибки.
- 02 Generic-цепочки могут «глушить» ошибки в неочевидных местах – стоит перепроверять.
- 03 as / type assertion – обходит компилятор, баги выживают.
- 04 Динамические свойства (obj[key]) – TS может не поймать.

КОД

// Сценарий рефакторинга

```
// было:
type User = { id: string; name: string };
function greet(u: User) { return 'Hi ' + u.name; }

// рефакторим: name → fullName
type User = { id: string; fullName: string };
// все usage u.name – ошибка компилятора
```

МОГУТ ДОСПРОСИТЬ

- 01 Что делает codemod?
- 02 Чем strict-mode помогает в рефакторинге?

03 Какие рефакторинги TS НЕ может поймать?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать, где жить shared-типам и как избегать дублирования DTO (Q1, Q3)

- ☐ [] Сравнить способы синхронизации FE/BE: monorepo / OpenAPI / GraphQL / tRPC / Zod (Q2)

- ☐ [] Объяснить, зачем runtime-валидация при наличии TS, и привести пример Zod (Q4, Q5)

- ☐ [] Развести domain model и API model, описать адаптер (Q6)

- ☐ [] Описать стратегию миграции с Flow на TS и поэтапного включения strict (Q7, Q8)

- ☐ [] Сформулировать, как бороться с as и какой TS-код считать плохим (Q9, Q11)

- ☐ [] Описать чек-лист TS-ревью (Q10)

- ☐ [] Описать backwards-compatible типы для API и работу с legacy (Q12, Q13)

- ☐ [] Назвать обязательный набор strict-флагов (Q14)

- ☐ [] Объяснить, чем TS помогает в рефакторинге (Q15)
